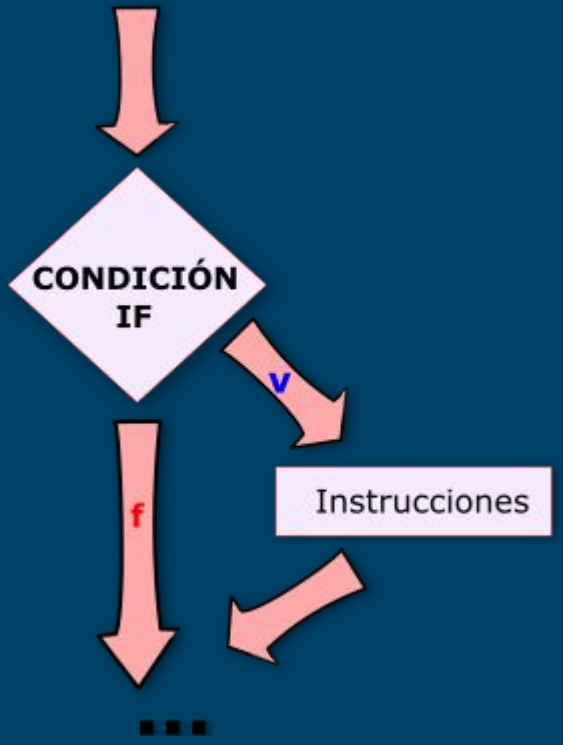
Instituto Educativo Económico Nacional
UEGP N°167

Estructuras de Decisión

Instituto Educativo Económico Nacional
UEGP N°167

1. Condicional simple
   - IF - ELSE - ELSEIF
2. Condicional múltiple
   - CASE o SWITCH
3. Operadores lógicos
   - And, OR



ESTRUCTURAS DE DECISIÓN

Definición

- Las sentencias de decisión o también llamadas de CONTROL DE FLUJO son estructuras de control que realizan una pregunta la cual retorna verdadero o falso, evaluando una condición, y selecciona la siguiente instrucción a ejecutar dependiendo la respuesta o resultado.

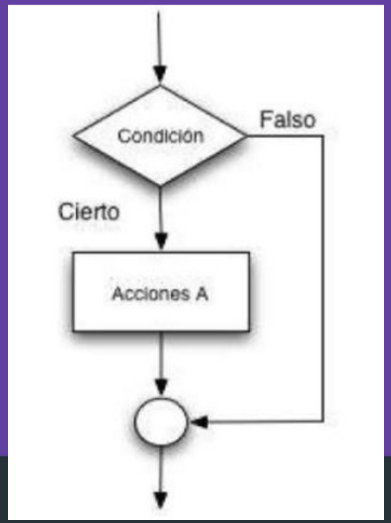
ESTRUCTURAS DE DECISIÓN

- Las instrucciones comienzan a ejecutarse de forma secuencial (en orden) y cuando se llega a una estructura condicional, la cual está asociada a una condición, se decide que camino tomar dependiendo siempre del resultado de la condición siendo ésta falsa o verdadera.

- Cuando se termina de ejecutar este bloque de instrucciones se reanuda la ejecución en la instrucción siguiente a la de la condicional.

ESTRUCTURAS DE DECISIÓN - CONDICIONAL SIMPLE

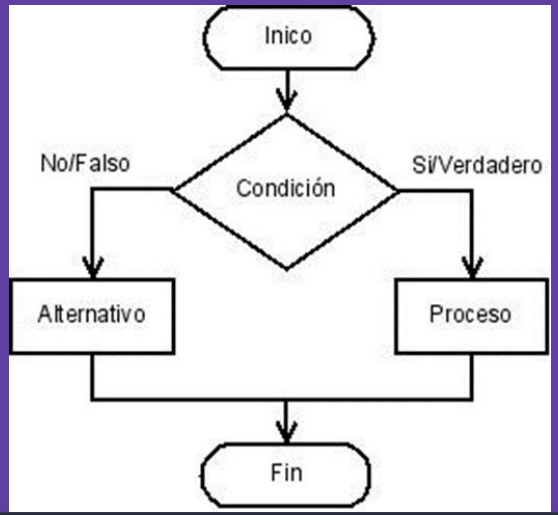
La instrucción **if** es, por excelencia, la más utilizada para construir estructuras de control.




```
if (condicion)
    {
        Set de instrucciones
    }
```

ESTRUCTURAS DE DECISIÓN - CONDICIONAL SIMPLE

A la instrucción If se le puede asociar una nueva secuencia para tratar los casos Falsos.




```
if (condicion)
    {
        Set de instrucciones     //PARTE VERDADERA
    }
else
    {
        Set de instrucciones 2   //Parte FALSA
    }
```

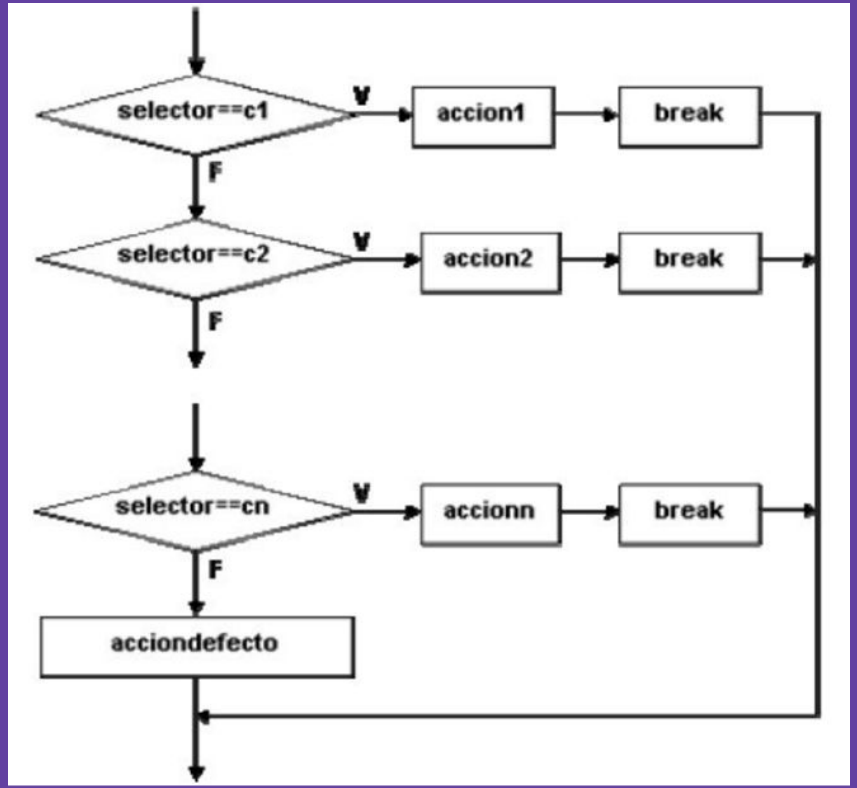
ESTRUCTURAS DE DECISIÓN - CONDICIONAL MÚLTIPLE

Switch es otra de las instrucciones que permiten la construcción de estructuras de control.

A diferencia de **if**, para controlar el flujo por medio de una sentencia switch se debe de combinar con el uso de las sentencias **case** y **break**.

Cualquier número de casos a evaluar por switch así como la sentencia default son opcionales.

La sentencia switch es muy útil en los casos de presentación de menús.




```
switch (condición)
{
    case primer_caso:
        bloque de instrucciones 1
    break;

    case segundo_caso:
        bloque de instrucciones 2
    break;

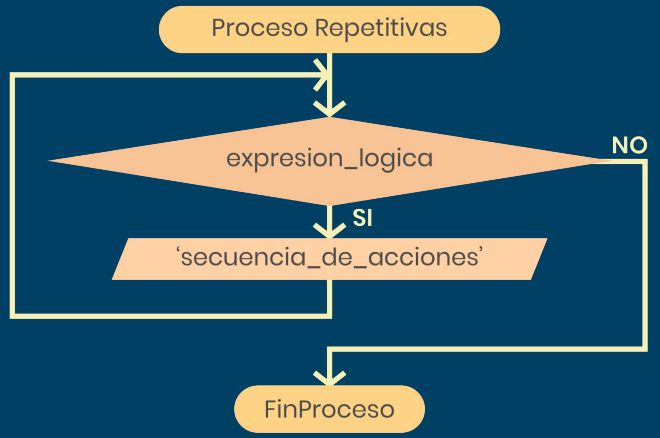
    case caso_n:
        bloque de instrucciones n
    break;

    default: bloque de instrucciones por defecto
}
```

Estructuras de Interacción

Instituto Educativo Económico Nacional
UEGP N°167

1. ¿Qué es una iteración?
2. FOR
3. WHILE
4. DO...WHILE
5. Contadores
6. Acumuladores

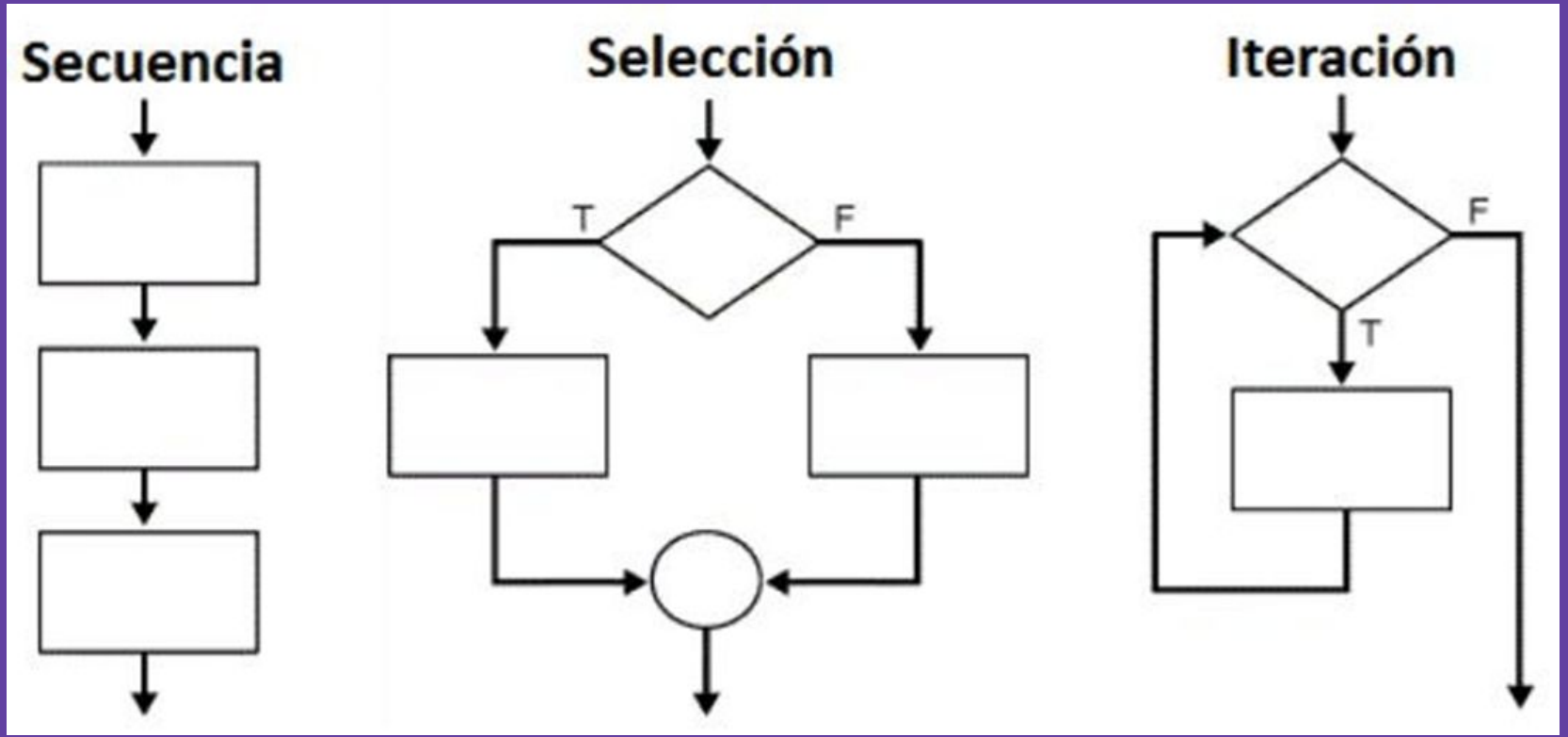


ESTRUCTURAS DE INTERACCIÓN

- Las Sentencias de Iteración o Ciclos son estructuras de control que repiten la ejecución de un grupo de instrucciones.

- Una sentencia de iteración es una estructura de control condicional, ya que dentro de la misma se repite la ejecución de una o más instrucciones mientras que una condición específica se cumpla.

- Muchas veces tenemos que repetir un número definido o indefinido de veces un grupo de instrucciones por lo que en estos casos utilizamos este tipo de sentencias.


Secuencia
Selección
T
F
Iteración
F
T

ESTRUCTURAS DE INTERACCIÓN - FOR

La sentencia for es útil para los casos en donde se conoce de antemano el número de veces que una o más sentencias han de repetirse.

```
for(contador; final; incremento)
{
    Codigo a Repetir;
}
```

CARACTERÍSTICAS ESTRUCTURA FOR

1.Contador es una variable numérica.

2. final es la condición que se evalúa para finalizar el ciclo (puede ser

independiente del contador)

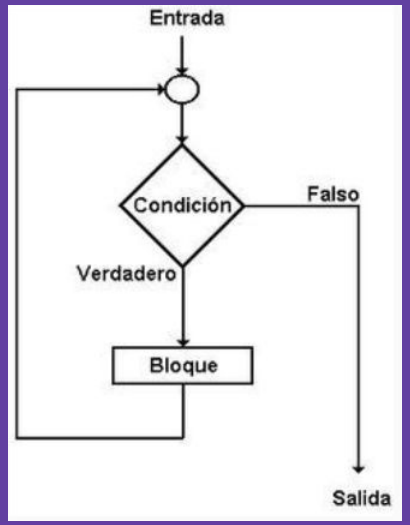
3. incremento es el valor que se suma o resta al contador.

4. el for evalúa la condición de finalización y mientras esta se cumpla continuarán las repeticiones.

```
for(contador; final; incremento)
{
    Codigo a Repetir;
}
```

ESTRUCTURAS DE INTERACCIÓN - WHILE

La sentencia while es útil en aquellos casos en donde no se conoce de antemano el número de veces que una o más sentencias se tienen que repetir.




```
while(condicion)
{
    código a Repetir
}
```

CARACTERÍSTICAS ESTRUCTURA WHILE

1. condición es la expresión a evaluar.

2. Si la condición es verdadera, continúa el ciclo de iteración.

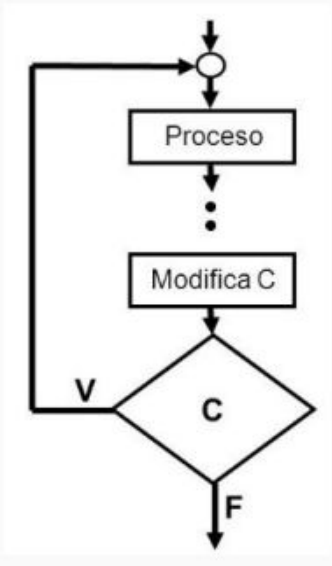
3. Si la condición inicialmente es falsa, nunca se ingresa al ciclo.

4. Evitar crear una condición que nunca se vuelva Falsa, ya que esto provocará un ciclo de iteración infinito, provocando que la aplicación falle por falta de memoria.

```
while(condicion)
{
    código a Repetir
}
```

ESTRUCTURAS DE INTERACCIÓN - DO .. WHILE

La sentencia do es usada generalmente en cooperación con while para garantizar que una o más instrucciones se ejecuten al menos una vez.




```
do
{
    código a Repetir
}
while(condicion)
```

CARACTERÍSTICAS ESTRUCTURA DO .. WHILE

1.condición es la expresión a evaluar.

2. Si la condición es verdadera, continúa el ciclo de iteración.

3. Se ejecuta al menos una vez el Código a Repetir, antes de evaluar la condición.

4. Evitar crear una condición que nunca se vuelva Falsa, ya que esto provocará un ciclo de iteración infinito, provocando que la aplicación falle por falta de memoria

```
do
{
    código a Repetir
}
while(condicion)
```

SENTENCIAS BREAK - CONTINUE

En la condición switch, la sentencia break es utilizada con el propósito de forzar un salto dentro del bloque switch hacia el final del mismo. ● La misma sentencia se puede usar un conjunto con las distintas operaciones de iteración. ● Además existe una nueva sentencia que se puede incluir en los ciclos iterativos: continue

SENTENCIAS BREAK

La sentencia break se usa para forzar un salto hacia el final de un ciclo controlado por for o por while.

La construcción del ciclo while para el caso de la sentencia break es diferente, esto para garantizar que el ciclo no vaya a caer en una iteración infinita.

```
for(contador; final; incremento)
{
   if(condicion) break;
   Codigo a Repetir;
}
```

```
while(condicion)
{
   if(condicion 2) break;
   código a Repetir
}
```

SENTENCIAS CONTINUE

La sentencia continue se usa para ignorar una iteración dentro de un ciclo controlado por for o por while.

La construcción del ciclo while para el caso de la sentencia continue es diferente, esto para garantizar que el ciclo no vaya a caer en una iteración infinita.

```
for(contador; final; incremento)
{
   if(condicion)continue;
   Codigo a Repetir;
}
```

```
while(condicion)
{
  if(condicion 2) continue;
  código a Repetir
}
```

Instituto Educativo Económico Nacional

CONTADORES

Un contador es una variable cuyo valor se incrementa o decrementa en una cantidad constante cada vez que se produce un determinado suceso o acción.

Los contadores se utilizan con la finalidad de contar sucesos o acciones internas de un bucle; deben realizar una operación de inicialización y posteriormente las sucesivas de incremento o decremento del mismo.

La inicialización consiste en asignarle al contador un valor y se situará antes y fuera del bucle.

```
int contador = 0;
while(contador<=10)
{
    contador = contador + 1;
}
```

```
int contador = 0;
while(contador<=10)
{
    contador++;
}
```

ACUMULADORES

Es una variable que suma sobre sí misma un conjunto de valores y de esta manera tener la suma de todos ellos en una sola variable.

La diferencia entre un contador y un acumulador es que mientras el primero va aumentando constantemente es decir de uno en uno, de dos en dos, de diez en diez, etc, el acumulador va aumentando en una cantidad variable.

```
int total = 0;
total = num + 1;
total = total + num;
total = total + 1;
```


11.11.13
BERLIN